

ModusToolbox™ Secure Request Framework user guide

About this document

Scope and purpose

This guide provides an overview of the ModusToolbox™ Secure Request Framework (SRF) and how to use it. This is intended to be an end-to-end guide covering not just the framework, but also assets it integrates with.

Intended audience

This is a high-level introduction to the SRF. It's intended as a beginner guide to understanding the many pieces and how they fit together. This guide expects familiarity with the ModusToolbox™ ecosystem, as well as arm®'s TrustZone feature and Cortex-M Security Extensions (CMSE) support.

Acronyms

- BSP – Board Support Package
- CMSE – Cortex-M Security Extensions
- DTCM – Data Tightly Coupled Memory
- IDAU – Implementation Defined Attribution Unit
- ioVec – Input/Output Vectors, similarly inVec/outVec
- IPC – Inter-processor Communication
- ITCM – Instruction Tightly Coupled Memory
- MPC – Memory Protection Controller
- NSC – Non-Secure Callable
- PDL – Peripheral Driver Library
- PPC – Peripheral Protection Controller
- PSA – Platform Security Architecture
- SAU – Security Attribution Unit
- SCB – Serial Communication Block
- SRF – Secure Request Framework

Table of Contents

Table of Contents

About this document.....	1
Acronyms.....	1
Table of Contents	2
1 Overview.....	3
2 Theory of operation.....	4
2.1 Request flow	4
2.2 Requests from non-TrustZone core.....	6
2.3 Input and output vectors	8
2.4 Input and output structures.....	9
2.5 Memory access validation.....	10
2.6 PPC integration	13
2.6.1 A brief on PDL integration.....	16
3 Creating operations.....	18
3.1 Modules and module registration	18
3.2 Submodules.....	19
3.3 Operations.....	21
3.3.1 Operation implementation.....	22
3.3.2 Permissions and resources	23
3.3.3 Non-secure argument validation.....	24
3.3.4 Operation arrays	25
4 Calling operations	27
4.1 Request pools and argument packing.....	27
4.2 Shared memory and DCache alignment	28
5 Customization	30
Revision history.....	32

Overview

1 Overview

The ModusToolbox™ SRF provides a reusable infrastructure for implementing multiple secure operations via a single non-secure-callable (NSC) entry point, which also performs basic argument validation. The design is generic with no dependency or knowledge of hardware peripherals or devices. The framework depends on arm®'s TrustZone feature. It supports calling across the security barrier of a single core, and also calling from a non-TrustZone core to a TrustZone core via an Inter-Processor Communication (IPC).

The SRF offers a standardized way to implement secure operations that are accessible to the non-secure world. It integrates arm's Cortex-M Security Extensions (CMSE) and Infineon's Peripheral Protection Controller (PPC) and Memory Protection Controller (MPC) regions into a single streamlined process.

Secure Infineon devices can be configured using PPC and MPC regions. The PPC defines the access permissions of a peripheral resource, such as a Serial Communication Block (SCB). The MPC is similar, but it is applicable for memory regions. It is akin to arm's Implementation Defined Attribution Unit (IDAU), but more granular. Together, these allow for a highly customizable security configuration for a given device.

However, one limitation of the PPC regions is granularity. For example, peripheral clock dividers (within a given PERI instance) are either all secured, or all non-secured. Consider for example if an application had a secured SCB instance and a non-secured PWM. Both of these are driven by different peripheral clock dividers. In this situation, all dividers would have to be secured to protect the SCB. However, it would leave the non-secure project unable to directly configure the PWM frequency.

This is where the SRF comes in; it allows the secure project to expose the needed functionality (e.g., the clock divider configuration) to the non-secure project and tie it to specific hardware instances. In the above example, the secure project would expose a "set divider" API tied to the PWM's clock divider (and potentially others), and hence enable the non-secure project to configure the PWM frequency despite its divider being secured.

Theory of operation

2 Theory of operation

The secure world exposes *operations* to the non-secure world. An operation is a function with strict argument requirements and *permission* arrays which define which peripherals it can be invoked on. These are grouped into *submodules* which are grouped into *modules*. A module is the unit of integration between different entities in the system that wish to expose secure operations. For example, a middleware library would be one module and submodules would be categories of functionality in that library. Modules are registered to a *secure context* structure.

The non-secure world invokes operations by submitting *requests*. Requests consist of input and output vectors, or *ioVecs*. The first inVec and first outVec must point to dedicated *input* and *output structures*. The SRF uses these to identify the operation and peripheral, perform validation, as well as store the result of the operation.

2.1 Request flow

The secure project is responsible for implementing operations and defining permissions, as well as initializing the SRF and registering the supported modules. Implementing an operation includes, defining various IDs and restrictions on arguments (e.g., size of expected input data, if applicable).

To execute these operations, the non-secure project needs to submit a request to the SRF. The SRF is responsible for validating all arguments and identifying the target operation. If the validation is successful, the operation is executed. A high-level sequence diagram of the process occurring in a single-core environment is included in Figure 1.

In the diagram, the Board Support Package (BSP) initialization routine performs all steps listed in the “Initialization” section during `cybsp_init()`. The only module registered by default is the PDL. Registering custom modules can be done in the secure project main function, prior to starting the non-secure project. Creation and registration of modules are covered in [Section 3.1](#).

Theory of operation

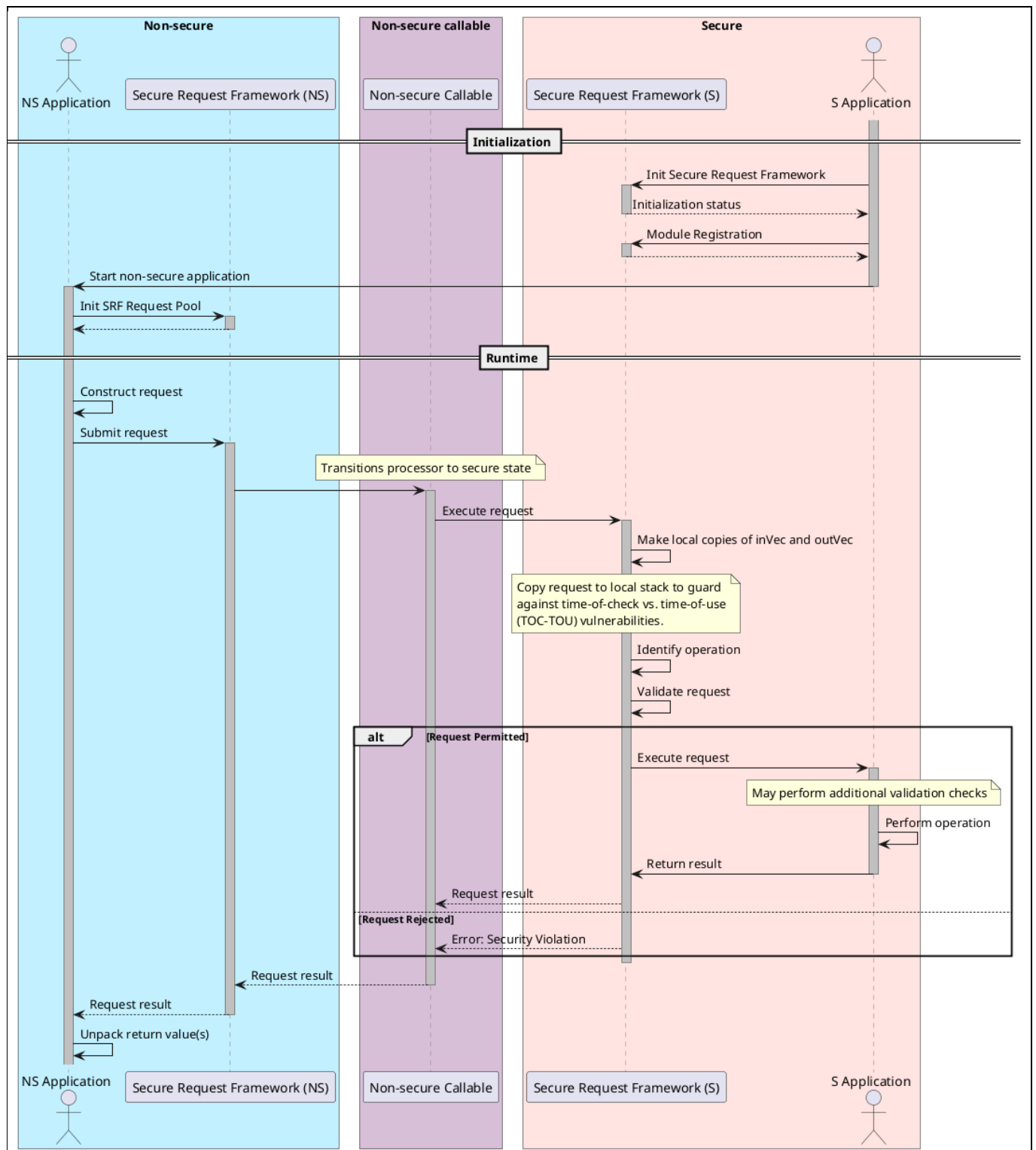


Figure 1 Request submission flow on a TrustZone core

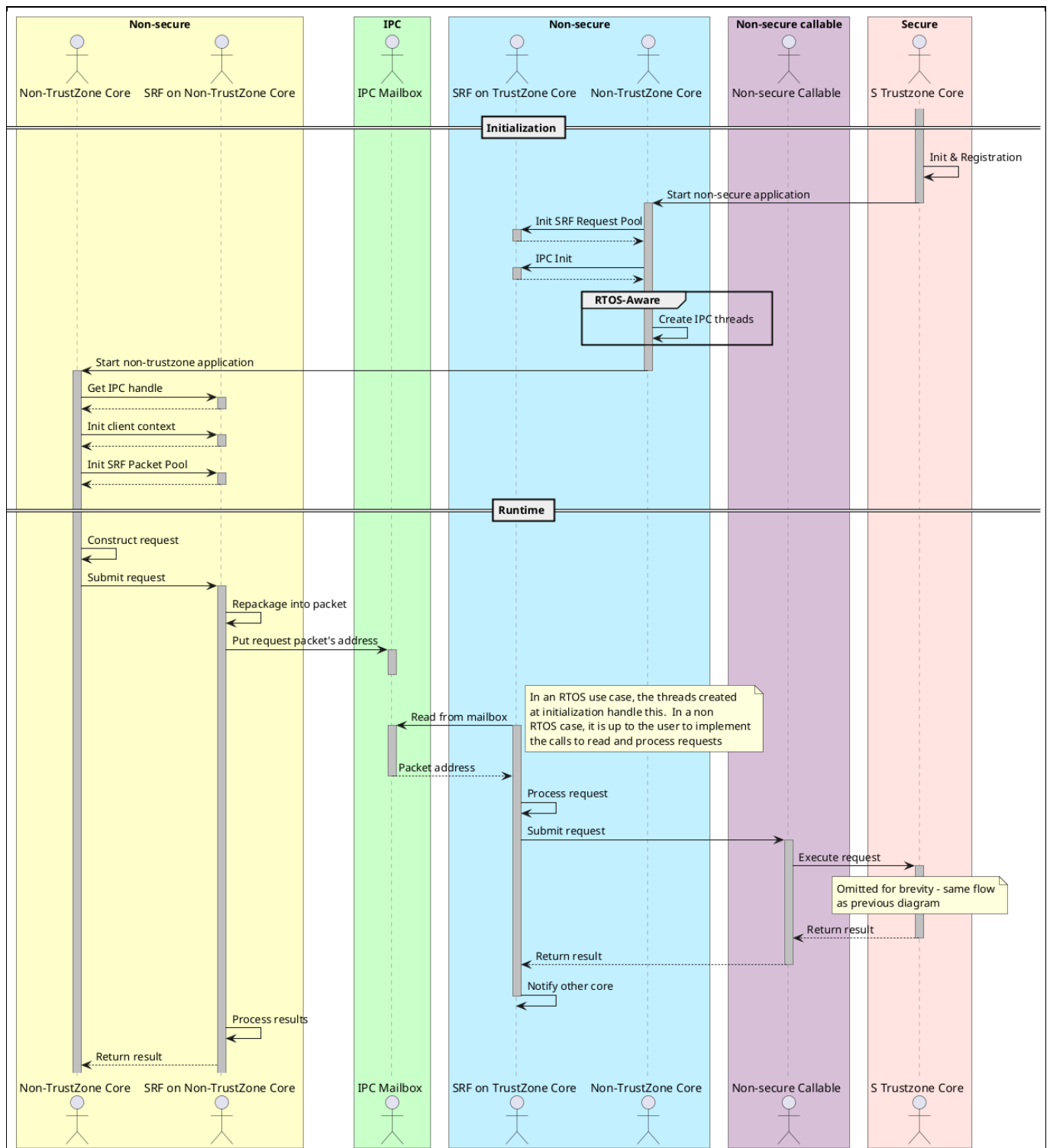
Theory of operation

2.2 Requests from non-TrustZone core

A feature of the SRF is that the process of submitting a request from any non-secure domain to the secure side is identical, regardless of the core. Both the non-secure side of a TrustZone enabled core and a different core are treated the same way (with one exception being data alignment when DCache is present and enabled. This is discussed further in [Section 4.2](#)). The SRF performs all additional steps to pass the request through IPC to the non-secure side of the TrustZone core so that it can then be passed through the NSC.

The multi-core initialization process is the same as the single-core flow; the BSP performs all the initialization steps except additional custom module registration.

Theory of operation



The IPC request submission flow requires more information than a cross-security boundary SRF call. The SRF will generate this information internally and group it with the standard SRF arguments into a packet. A packet is defined as an `mtb_srf_ipc_packet_t` struct as in the example below (`_MTB_SRF_DATA_ALIGN` is discussed further in [Section 4.2](#)):

Theory of operation

```

001     typedef struct _MTB_SRF_DATA_ALIGN
002     {
003         // Pointer to input vectors
004         mtb_srf_invec_ns_t*  inVec_ptr;
005         // Number of input vectors in inVec_ptr array
006         uint8_t              inVec_cnt_ns;
007         // Pointer to output vectors
008         mtb_srf_outvec_ns_t* outVec_ptr;
009         // Number of output vectors in the outVec_ptr array
010         uint8_t              outVec_cnt_ns;
011         // Index in the semaphore array for semaphore used to
012         // signal completion
013         uint16_t             semaphore_idx;
014         // Result of the SRF operation, which may not be success
015         // even if the IPC request succeeded
016         cy_rslt_t            request_rslt;
017     } mtb_srf_ipc_packet_t;

```

Unlike the single-core flow, the multi-core flow requires additional effort when used in a non-RTOS environment. In an RTOS environment, the BSP creates threads that read from the mailbox and process the requests automatically. In a non-RTOS environment, it is the user's responsibility to set up their non-secure TrustZone-enabled core's project to do so. At its simplest, this could be a loop that process the requests in a round robin.

2.3 Input and output vectors

The SRF submission accepts four arguments: an input vector array with a count, and an output vector array with a count. These are called ioVecs, and they are used to contain all data passed between security states. These are consistent with the ioVec types defined by arm as part of the PSA. The ioVecs are defined as follows:

```

001     typedef struct
002     {
003         // Start address of the memory buffer
004         const void*      base;
005         // Size of memory buffer in bytes
006         size_t           len;
007     } mtb_srf_invec_ns_t;
008
009     typedef struct
010     {
011         // Start address of the memory buffer
012         const void*      base;
013         // Size of memory buffer in bytes
014         size_t           len;
015     } mtb_srf_outvec_ns_t;

```

These are placed in a continuous section of memory. They could be buffers or structures or any other type. The SRF only cares about the addresses of the memory; namely if it is accessible to both the NS and S CPU. More details on memory addresses can be found in [Section 2.5](#). An input vector contains any data that the SRF or operation should read. An output vector contains locations for the SRF or operation to write results to.

Theory of operation

The maximum number of ioVecs that can be passed to the SRF (sum of inVec count and outVec count) is defined by `MTB_SRF_MAX_IOVEC`. This defaults to 4 to satisfy PSA requirements, but can be overridden by defining a custom value for this macro via Makefile defines.

2.4 Input and output structures

The first inVec and first outVec are each reserved. These must point to an `mtb_srf_input_ns_t` and `mtb_srf_output_ns_t` structure, respectively. The purposes of these two structures are threefold:

- The input structure contains a request structure, `mtb_srf_request_ns_t`, which the SRF uses to identify the target operation and peripheral resource.
- Both input and output structures also contain flexible arrays for non-pointer arguments. This allows easy passing of individual values without using extra ioVecs.
- The output structure contains the operation return value. The SRF will return its own error or success result directly. The caller should unpack the operation result to determine its return.

The three structures described are defined like so:

```

001     typedef struct
002     {
003         // ID of the module containing the requested operation
004         uint8_t      module_id;
005         // ID of the submodule containing within the module
006         uint8_t      submodule_id;
007         // ID of the requested operation within the submodule
008         uint8_t      op_id;
009         // Register base address for the operation
010         void*        base;
011         // Sub-block index for this operation
012         uint32_t     sub_block;
013     } mtb_srf_request_ns_t;
014
015     typedef struct
016     {
017         // Request information
018         mtb_srf_request_ns_t  request;
019         // Size of the input_values array
020         size_t                len;
021         // Flexible size array for non-pointer input arguments
022         uint8_t               input_values[];
023     } mtb_srf_input_ns_t;
024
025     typedef struct
026     {
027         // Result of the operation
028         cy_rslt_t      result;
029         // Size of the output_values array
030         size_t         len;
031         // Flexible size array for non-pointer output arguments
032         uint8_t        output_values[];
033     } mtb_srf_output_ns_t;

```

Theory of operation

An arbitrary number of modules can be registered with the SRF. Each module can contain many sub-modules, and each sub-module can define various operations. The various IDs defined in the request structure is addressed further in [Section 3](#).

The base and sub-block arguments in `mtb_srf_request_ns_t` are used to identify the peripheral that the operation should apply to (if applicable). For example, a timer read operation may use the base TCPWM1 and sub-block 6 to indicate TCPWM[1] Group[0] Counter[6]. A resource like an SCB would only need the base SCB5 to indicate which resource to use (the sub-block would be zero). If a resource only has one instance and no base address, then the base may be declared NULL and the sub-block zero.

The base arguments are the base addresses of the peripherals in memory, as accessed by the CPU. These are defined in the device header files. Below shows an example of LPCOMP's non-secure base address. Note that there is both a secure and non-secure version of this file, with different addresses.

```

001      /*****
002      *                                LPCOMP
003      *****/
004
005      #define LPCOMP_BASE 0x42880000UL
006      #define LPCOMP      ((LPCOMP_Type*)LPCOMP_BASE) /*0x42880000*/

```

The SRF automatically converts any NS peripheral base address passed in as the `mtb_srf_request_ns_t`'s `base` member into a S address alias. This is not done for the remaining `ioVec` base addresses (referring to `ioVec.base`). If conversion is needed, the macros `GET_ALIAS_ADDRESS` and `GET_NSALIAS_ADDRESS` are available. The first will adjust addresses in accordance with the SAU to match the security state of the caller (an S address will be masked to NS when called in a non-secure CPU state, and vice versa). The latter masks all addresses to NS. Attempting to access a peripheral from the wrong security state address will cause a fault.

A note on return types – the SRF returns its own result directly from the request submission. The operation result is stored in the output structure's `result` member. The SRF has five error codes it can return:

- `MTB_SRF_ERR_BAD_PARAM` – One or more arguments is invalid. This could be too many `ioVecs`, wrong size of flexible array, wrong `ioVec` counts. Any parameter that does not match the requirements listed by the operation structure
- `MTB_SRF_ERR_MODULE_ID_INVALID` – The module ID was not valid (greater than the max), not registered, or the submodule ID was out of bounds inside the module.
- `MTB_SRF_ERR_UNKNOWN_OPERATION` – The module was found, but the operation was not found.
- `MTB_SRF_ERR_SECURITY_POLICY_VIOLATION` – Either the `ioVecs` pointed to secure memory, or the request's target peripheral is not allowed by the operation's permissions.
- `MTB_SRF_ERR_ARGBUFF_FULL` – The argument cache cannot fit all the `ioVecs` marked as cacheable.

2.5 Memory access validation

The MPC defines which memory regions are considered secure. If no MPC is present, e.g., for ITCM/DTCM on PSOC™ Edge devices, then the region is always considered non-secure. This can be configured in the device configurator, which is then converted into arrays that the SRF uses to check the security state of the memory regions. When validating the `ioVecs`, the SRF checks only that the memory is accessible by the NS domain. The below screenshots of the device configurator demonstrate how to configure this.

Theory of operation

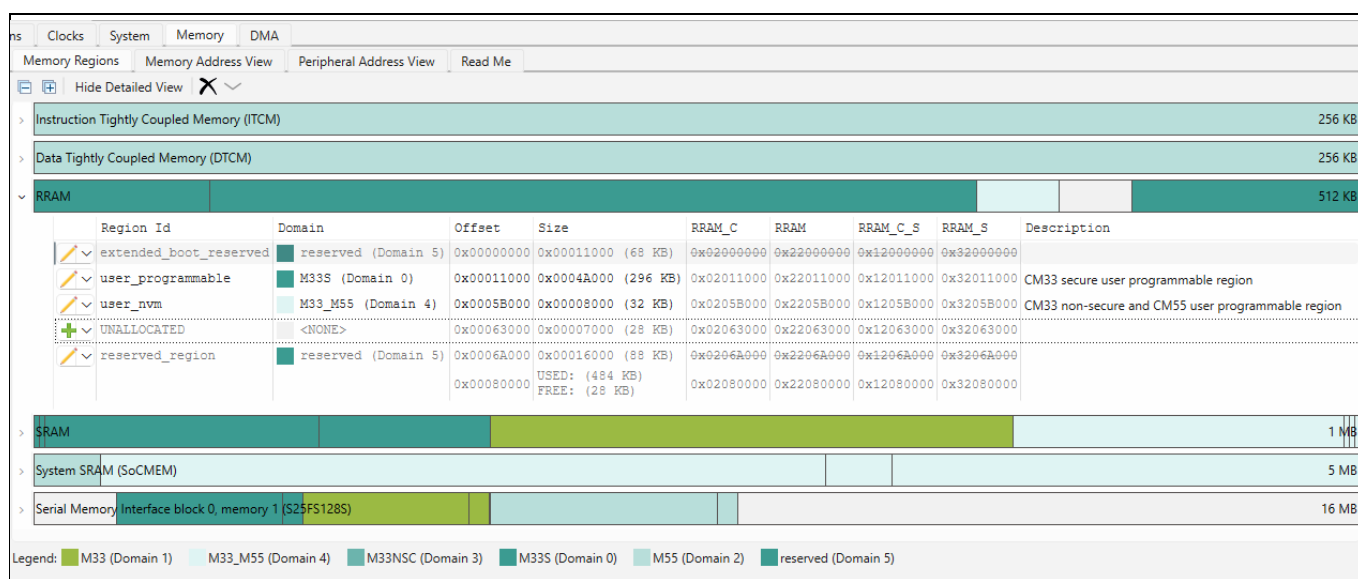


Figure 3 Memory Address View in Device Configurator

Figure 3 shows the RRAM region of a PSoC™ Edge device. It has a TrustZone-enabled CM33 (split into CM33_S and CM33_NS) and a CM55 core. Sections of memory are assigned a Domain. Only 3 domains are used for this RRAM configuration, but there are six domains available. These are listed in the Legend at the bottom.

- M33S for CM33_S
- M33 for both the CM33_NS and CM33_S
- M55 for CM55
- M33NSC for the CM33's NSC
- M33_M55 for memory accessible to both CM33 and CM55
- Reserved for secured regions, which cannot be configured

In brief, M33S or Domain 0 is the secure domain, and all other domains are non-secure.

The greyed-out regions are locked and cannot be configured, but the remaining memory is customizable.

Theory of operation

Under the size column, this example is using only 484 KB of the 512 KB region. To configure a region, click on the pencil icon to open a drop-down menu, and select Edit:

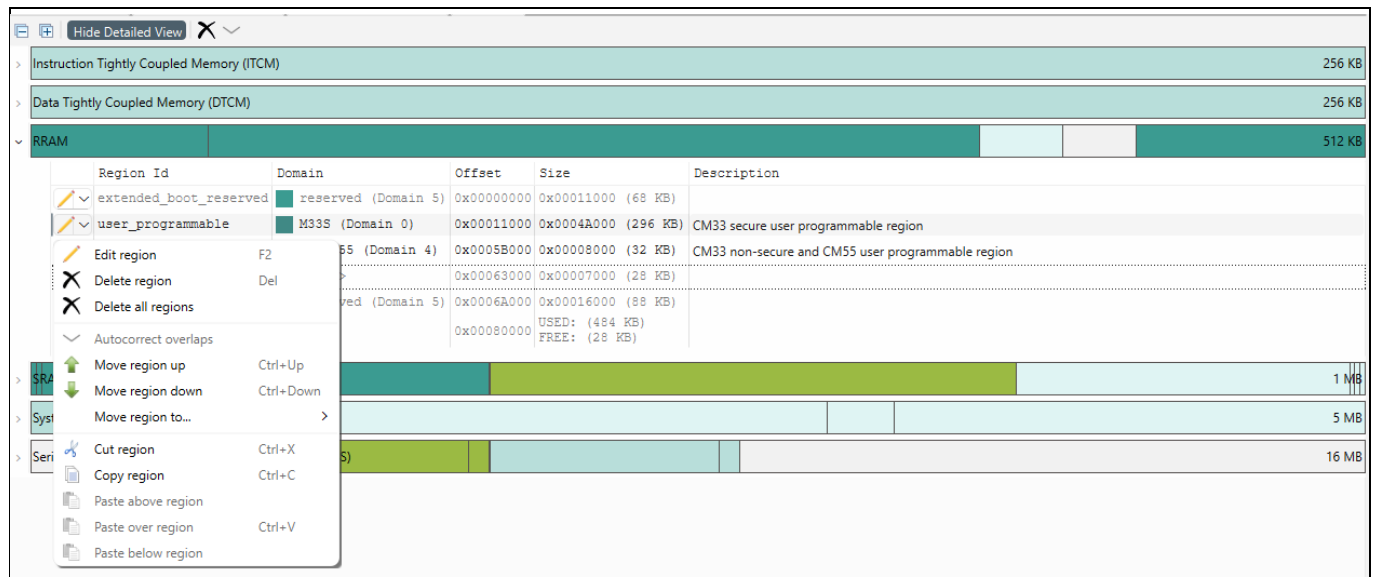


Figure 4 Opening memory region configuration menu

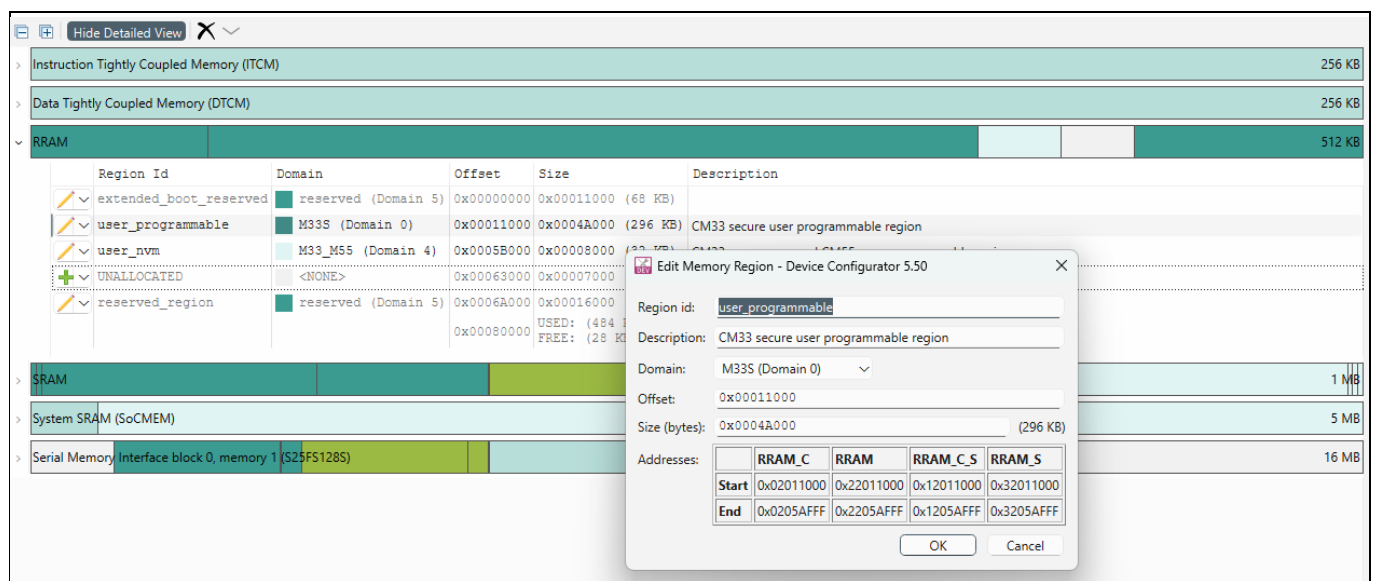
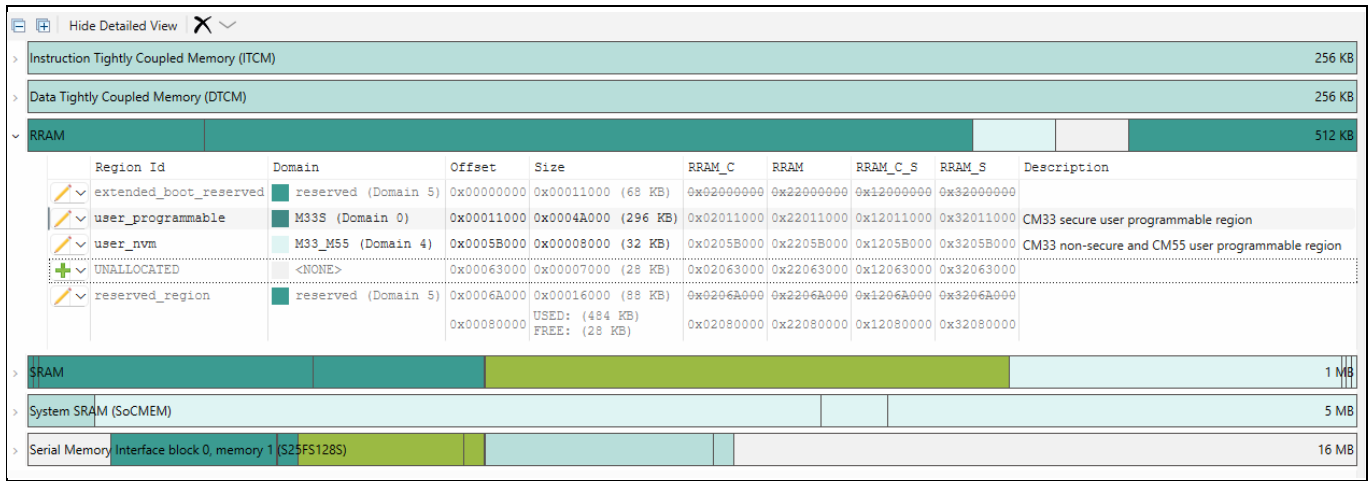


Figure 5 Memory region configuration options

Theory of operation

You can configure the size of the user programmable region. This menu shows additional address information. This information can be added to the top-level display via the 'Hide Detailed View' button at the top:



Region Id	Domain	Offset	Size	RRAM_C	RRAM	RRAM_C_S	RRAM_S	Description
extended_boot_reserved	reserved (Domain 5)	0x00000000	0x00011000 (68 KB)	0x02000000	0x22000000	0x12000000	0x32000000	
user_programmable	M33S (Domain 0)	0x00011000	0x0004A000 (296 KB)	0x02011000	0x22011000	0x12011000	0x32011000	CM33 secure user programmable region
user_nvm	M33_M55 (Domain 4)	0x0005B000	0x00008000 (32 KB)	0x0205B000	0x2205B000	0x1205B000	0x3205B000	CM33 non-secure and CM55 user programmable region
UNALLOCATED	<NONE>	0x00063000	0x00007000 (28 KB)	0x02063000	0x22063000	0x12063000	0x32063000	
reserved_region	reserved (Domain 5)	0x0006A000	0x00016000 (88 KB)	0x0206A000	0x2206A000	0x1206A000	0x3206A000	
			USED: (484 KB) FREE: (28 KB)	0x02080000	0x22080000	0x12080000	0x32080000	

Figure 6 RRAM regions with Detailed View mode enabled

Only non-reserved regions are converted into code. These two regions will appear like so:

```

001      const mtb_srf_protection_range_s_t
      mxrramc_0_mpc_0_srf_protection_range_s[mxrramc_0_mpc_0_REGION_COUNT] =
002      {
003          {
004              .start = (void *) 0x22011000,
005              .length = 0x4A000U,
006              .is_secure = true,
007          },
008          {
009              .start = (void *) 0x2205B000,
010              .length = 0x8000U,
011              .is_secure = false,
012          },
013      };

```

The user_programmable region is in the M33S domain, which is only accessible to the CM33_S (secured). The other region is accessible to both CM33 and CM55 (not secured). If an ioVec was passed to the SRF and is in the secured region, the SRF will reject the request with a security violation error.

For further reading, see the [Device Configurator user guide](#).

2.6 PPC integration

The PPC is responsible for ensuring that peripherals are only accessed by the CPUs in matching security states. That is, should SCB2 be secured, only a secured state core could access it. The opposite is only partially true; a secured state core cannot access a non-secured SCB2 through the secure address. It can access a non-secured SCB2 through the non-secure address. Attempting to access a resource that doesn't match the security state will generate a fault. The SRF does not check PPC regions as part of run-time validation, but they are relevant for determining when to use SRF.

Theory of operation

Peripherals do not always map one-to-one with PPC regions. One or more peripherals may use a region, and one peripheral may use many regions. It's recommended to use the Device Configurator to configure regions as they group by peripheral(s). The following demonstrates how to do so on a PSoC™ Edge device:

The PPC configuration is under the **System** tab in the Device Configurator.

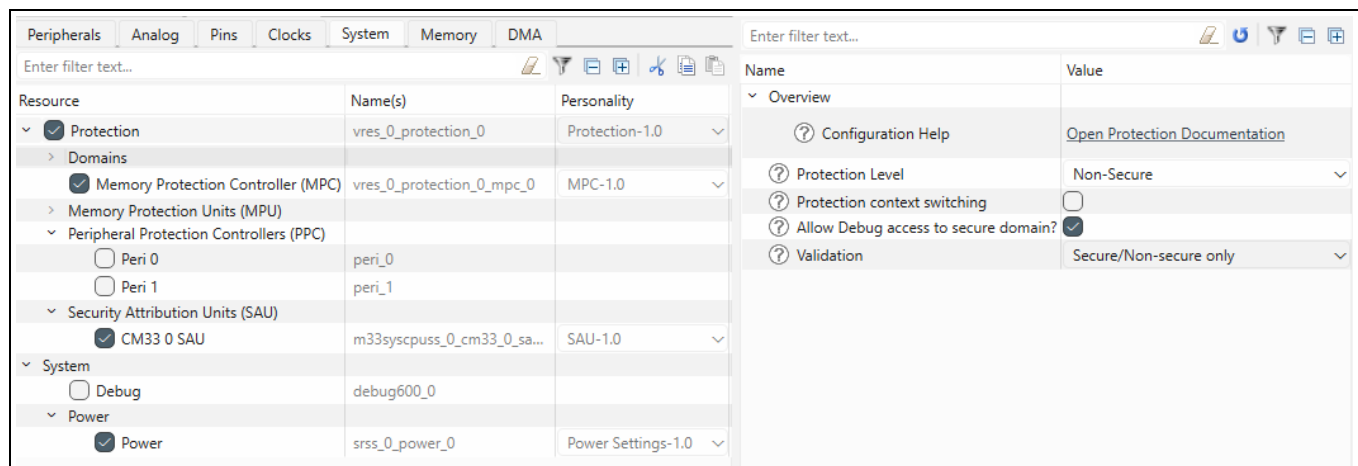


Figure 7 System tab of the Device Configurator

This example starts with the PPC disabled. Attempting to enable a PPC here will generate a warning. You must first configure the **Protection Level**.

Click on the **Protection** resource on the left to show the **Parameters** on the right. From here, select the desired **Protection Level** from the pull-down menu:

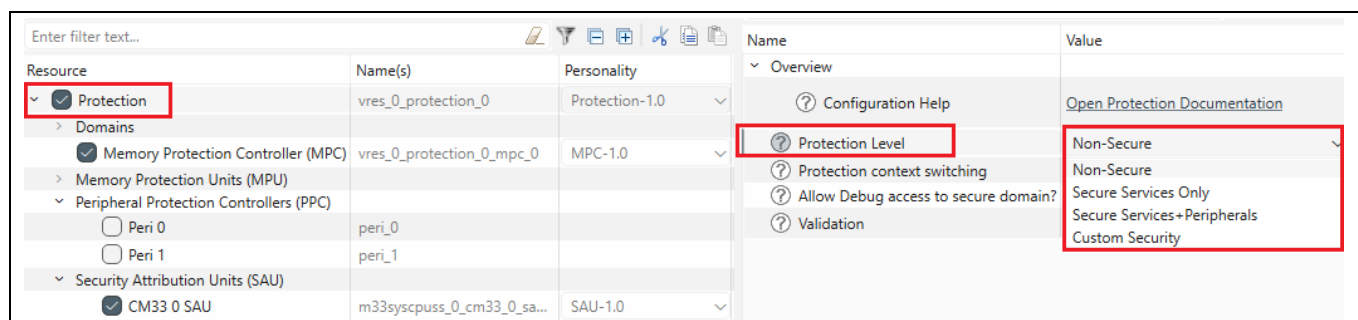


Figure 8 Protection Level configuration options

Both 'Secured Services + Peripherals' and 'Custom Security' will allow PPC customization. We recommend using 'Secured Services + Peripherals' when starting. The 'Custom Security' option is for advanced users. It allows fine-tune control over PPC regions by allowing each region to be configured directly, but it also disables the system-level consistency checking. The following figures will demonstrate the 'Secured Services + Peripherals' route.

Once either is selected, a PPC may be enabled. There are two PPCs as different regions map to different PPC instances.

Theory of operation

In Figure 9, only the first PPC is enabled. Enabling and selecting it will open the configuration menu.

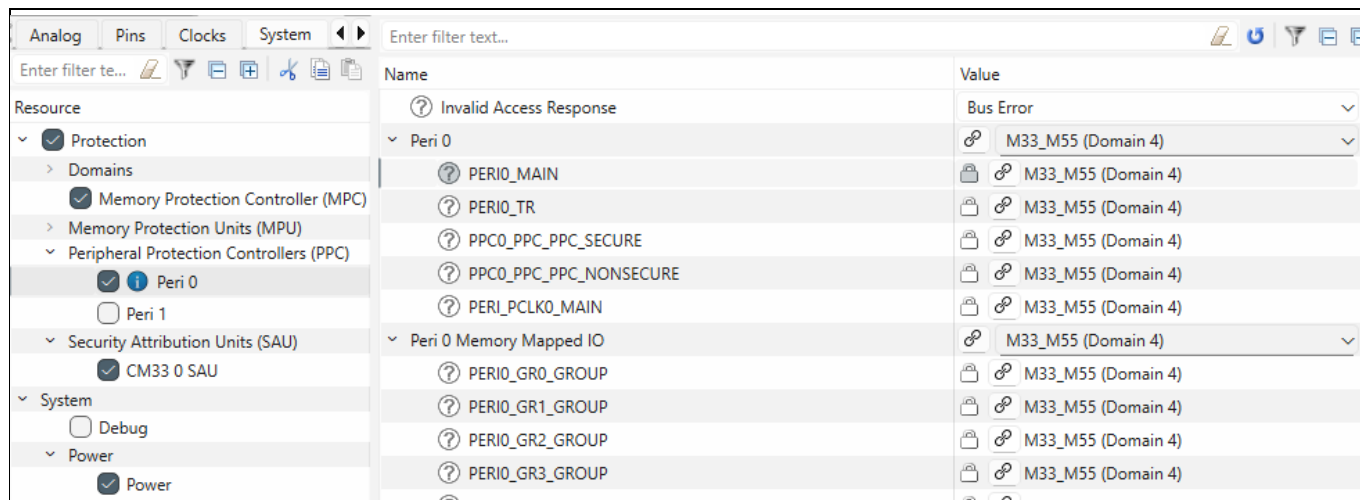


Figure 9 PPC instance 0 enabled, showing PPC regions grouped by related peripheral

Related regions are grouped together under one name and all share a security domain. See [Section 2.5](#) for explanation of security domains. All regions have defaulted to M33_M55, so they are non-secure and accessible to both cores. To secure SCB10, select M33S (Domain 0).

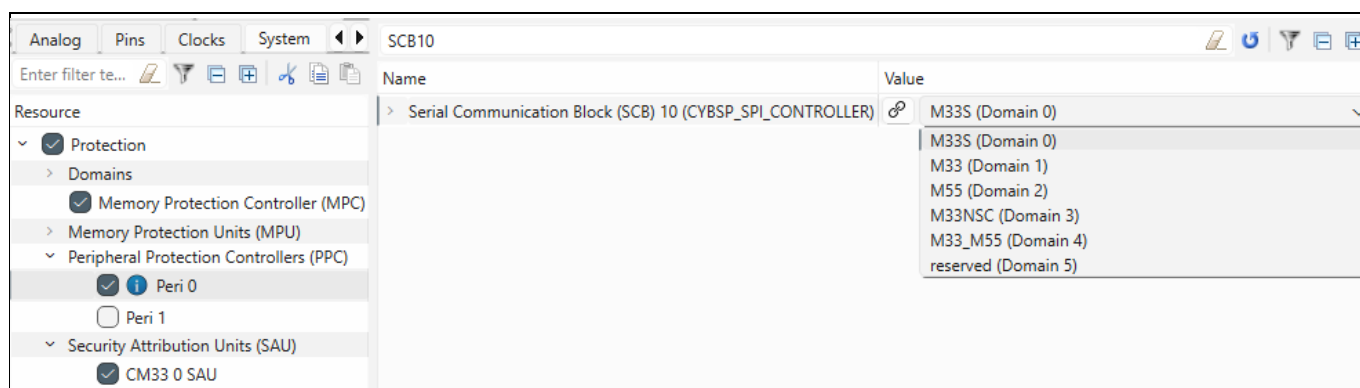


Figure 10 Setting SCB10 peripheral to Secure

Now there is one secured region. However, the SCB interacts with other peripherals, like GPIOs and clocks. The configurator warns of locations where two peripherals are connected but cross a security boundary.

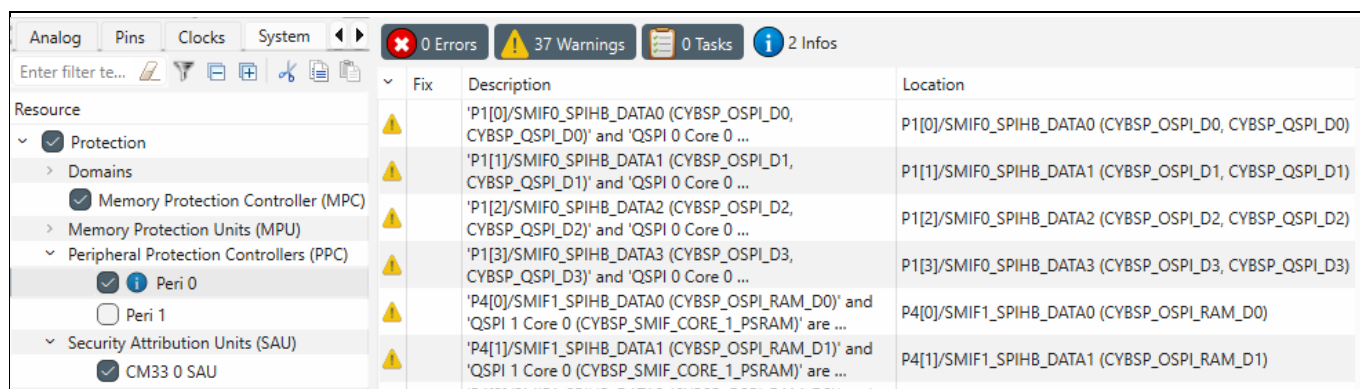
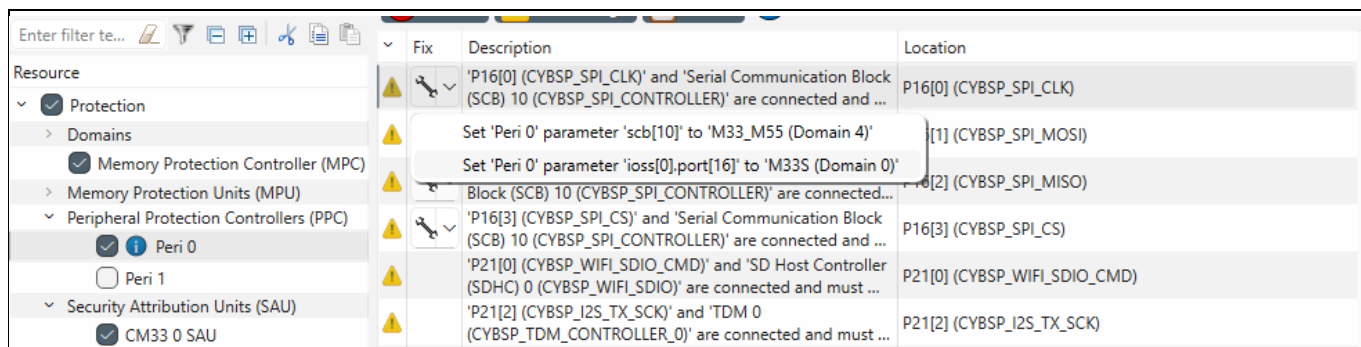


Figure 11 Warnings about all resource connections which cross security boundaries

Theory of operation

Many of these have suggested fixes, e.g., this GPIO pin suggests to either make SCB10 non-secure, or secure the GPIO pin:

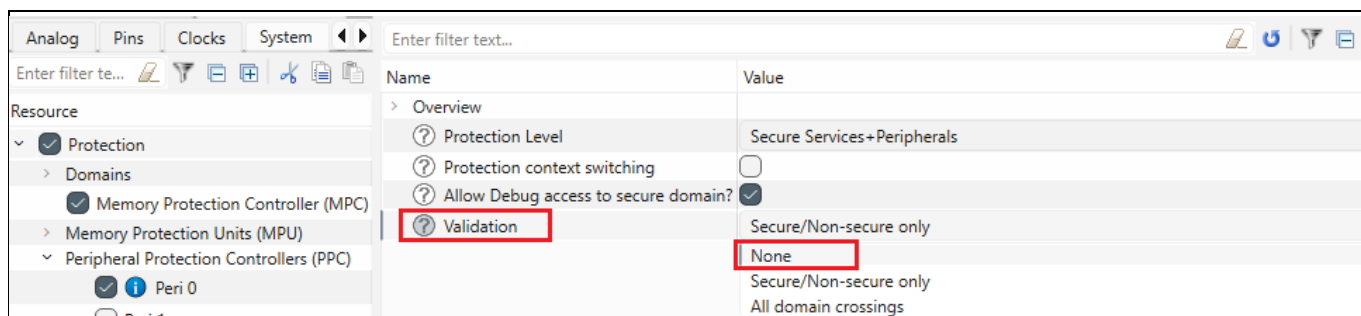


Fix	Description	Location
	'P16[0] (CYBSP_SPI_CLK)' and 'Serial Communication Block (SCB) 10 (CYBSP_SPI_CONTROLLER)' are connected and ...	P16[0] (CYBSP_SPI_CLK)
	Set 'Peri 0' parameter 'scb[10]' to 'M33_M55 (Domain 4)'	[1] (CYBSP_SPI_MOSI)
	Set 'Peri 0' parameter 'ioss[0].port[16]' to 'M33S (Domain 0)'	[2] (CYBSP_SPI_MISO)
	Block (SCB) 10 (CYBSP_SPI_CONTROLLER)' are connected...	
	'P16[3] (CYBSP_SPI_CS)' and 'Serial Communication Block (SCB) 10 (CYBSP_SPI_CONTROLLER)' are connected and ...	P16[3] (CYBSP_SPI_CS)
	'P21[0] (CYBSP_WIFI_SDIO_CMD)' and 'SD Host Controller (SDHC) 0 (CYBSP_WIFI_SDIO)' are connected and must ...	P21[0] (CYBSP_WIFI_SDIO_CMD)
	'P21[2] (CYBSP_I2S_TX_SCK)' and 'TDM 0 (CYBSP_TDM_CONTROLLER_0)' are connected and must ...	P21[2] (CYBSP_I2S_TX_SCK)

Figure 12 The configurator will make recommendations to resolve warnings

The remaining warnings that do not have suggested fixes list their PPC regions, which need to search for to resolve. One warning lists the `CYBSP_SMIF_CORE_0_XSPI_FLASH` region. This doesn't show under PPC instance 0, but enabling instance 1 and searching for it allows us to set its domain to 0 too. Enabling PPC instance 1 resolves the majority of our warnings, and leaves only a few remaining that are all resolved via suggested fixes.

If desired, you can disable the warnings under the Protection configuration's **Validation** option by selecting 'None.'



Name	Value
Overview	
Protection Level	Secure Services+Peripherals
Protection context switching	☐
Allow Debug access to secure domain?	☒
Validation	Secure/Non-secure only
	None
	Secure/Non-secure only
	All domain crossings

Figure 13 Disabling security boundary warnings

For further reading, see the [Device Configurator user guide](#).

2.6.1 A brief on PDL integration

The PDL has several Secure Aware drivers. These drivers have APIs marked as Secure Aware, which indicates that they will automatically call to the SRF to perform the operation when appropriate (the hardware resource is secured and the API is called from a non-secure CPU state). Not all APIs supported by a driver are Secure Aware; only a select few were specified as they were a common use case. For example, SysCLK supports Secure Aware read and write for peripheral clock dividers, but not for the ECO. The Secure Aware PDL only handles non-secure projects calling on secure resources, not the inverse. The application may generate a CMSE non-secure function pointer to perform this action.

Note: Secure Aware PDL Drivers are unrelated to the hardware Secure Aware Peripherals.

The PDL module is registered by default in the BSP's `cybsp_init()` by the secure project. When a Secure Aware PDL API is called from a non-secure project on a secure resource, the PDL will redirect it through the SRF.

Theory of operation

It allocates a request from the shared pool, populates it, and submits it. After validation and look-up, the SRF calls to the PDL to perform the operation from the secure side. The operation calls the original API again, but this time it will skip the SRF redirect and perform the requested action.

This Secure Aware functionality is enabled by securing the matching PPC region. Some drivers use multiple regions. These drivers require the regions have matching PPC configurations to ensure functionality.

After configuring PPC regions, a Secure Aware PDL driver may generate a build error like this:

```
In file included from ../../mtb_shared/mtb-dsl-pse8xxgp/release-v1.0.0/pdl/drivers/include/cy_pdl_srf.h:132,
from ../../mtb_shared/mtb-dsl-pse8xxgp/release-v1.0.0/pdl/drivers/include/cy_syspm.h:1627,
from ../../mtb_shared/mtb-dsl-pse8xxgp/release-v1.0.0/pdl/drivers/include/cy_scb_common.h:365,
from ../../mtb_shared/mtb-dsl-pse8xxgp/release-v1.0.0/pdl/drivers/include/cy_scb_ezi2c.h:252,
from ../../mtb_shared/mtb-dsl-pse8xxgp/release-v1.0.0/pdl/devices/include/cy_pdl.h:37,
from C:/Users/StitesZ/mtb_proj/screenshots/mtb_shared/mtb-dsl-pse8xxgp/release-v1.0.0/device-utils/syspm/so
urce/mtb_syspm_callbacks_scb_uart.c:27:
../../mtb_shared/mtb-dsl-pse8xxgp/release-v1.0.0/pdl/drivers/include/cy_syspm_srf.h:76:6: error: #error "Unsupported PPC reg
ions' security state: SRSS_MAIN, SRSS_HIB_DATA, PWRMODE_PWRMODE, and M55APPCPUSS PPC regions must all be configured to the s
ame security state"
 76 | #error "Unsupported PPC regions' security state: SRSS_MAIN, SRSS_HIB_DATA, PWRMODE_PWRMODE, and M55APPCPUSS PPC
regions must all be configured to the same security state"
```

Figure 14 Build error coming from SysPM, complaining about PPC region settings.

There are two options to resolve these errors:

- Update the PPC regions to match as the error requests
- Disable the Secure Aware feature of that driver

Each driver has a matching makefile define that can be used to disable it. For SysPM:

```
001     #if !defined(CY_PDL_ENABLE_SECURE_AWARE_SYSPM)
002     /** SYSPM Secure Aware Driver enablement. Set to 0 via
003     * Makefile defines to disable. When not manually set,
004     * defaults to the PDL-wide Secure Aware Driver enablement
005     * status. */
006     #define CY_PDL_ENABLE_SECURE_AWARE_SYSPM
(CY_PDL_ENABLE_SECURE_AWARE)
007     #endif /* !defined(CY_PDL_ENABLE_SECURE_AWARE_SYSPM) */
```

These default to CY_PDL_ENABLE_SECURE_AWARE. This is another makefile define, but will disable all PDL driver Secure Awareness.

Further information on the Secure Aware PDL can be found in the Device Support Library documentation, under API Reference > PDL API Reference > PDL SRF. This section includes list of all the Secure Aware Drivers – see each driver’s enumeration documentation for lists of all APIs included.

Creating operations

3 Creating operations

The secure project is responsible for defining and implementing the operations for the non-secure project. This section discusses the various steps and pieces that are required.

3.1 Modules and module registration

A module is an SRF object. It contains an array of sub-modules. A sub-module is an array of operations. A module is defined via the `mtb_srf_module_s_t` structure.

```

001     typedef struct
002     {
003         // ID for this module. Must be globally unique
004         uint8_t      module_id;
005         // Array of operations in this module/submodule
006         mtb_srf_op_s_t** op_by_submod;
007         // Length of each entry of the `op_by_submod` array
008         size_t*      num_op_by_submod;
009         // Length of the `num_op_by_submod` array
010         size_t      num_submod;
011     } mtb_srf_module_s_t;

```

The module ID is simple index starting from 0. The BSP contains a default module ID for user applications to use called `MTB_SRF_MODULE_USER`. This is defined in the BSP file `mtb_srf_config.h`. It contains all user-owned SRF configuration options. Here's a snippet of a BSP that contains two default module IDs – the PDL and the User:

```

001     /** Number of modules */
002     #define MTB_SRF_MODULE_COUNT                (2U)
003
004     /** Module for PDL operations */
005     #define MTB_SRF_MODULE_PDL                  (0U)
006     /** Module for User operations */
007     #define MTB_SRF_MODULE_USER                 (1U)

```

The user module is not registered and contains no sub-modules or operations by default. It is recommended to use this as a starting point to add operations. If another module is required, simply define a new module ID and increment the `MTB_SRF_MODULE_COUNT`.

All module IDs must be globally unique and are centrally defined through the file `mtb_srf_config.h`, which is owned by the application. Libraries that expose operations through the SRF consume this file to determine the module ID which they should register. If a library requires a module ID but one is not provided, it will provide a compile time error to indicate that one must be added to this file.

Module IDs function as indexes in the array of size `MTB_SRF_MODULE_COUNT` which the SRF uses when searching for operations.

A module may be registered via the `mtb_srf_module_register` API. This accepts an SRF context and the module. The BSP offers a default context structure, `cybsp_srf_context`, which is available for use. Each context structure can contain up to `MTB_SRF_MODULE_COUNT` modules. It is initialized and the PDL module is registered by default during `cybsp_init`:

Creating operations

```
001     #if !defined(CYBSP_DISABLE_SRF_INIT)
002     /* Set up the SRF and register the PDL module */
003     result = mtb_srf_init(&cybsp_srf_context);
004     if (result != CY_RSLT_SUCCESS)
005     {
006         return result;
007     }
008     #if defined(CY_PDL_ENABLE_SECURE_AWARE) && \
009         (CY_PDL_ENABLE_SECURE_AWARE)
010     result = cy_pdl_srf_module_register(&cybsp_srf_context);
011     if (result != CY_RSLT_SUCCESS)
012     {
013         return result;
014     }
015     #endif /* defined(CY_PDL_ENABLE_SECURE_AWARE) &&
016            (CY_PDL_ENABLE_SECURE_AWARE) */
017     #endif /* !defined(CYBSP_DISABLE_SRF_INIT) */
```

Refer to more details on the `CYBSP_DISABLE_SRF_INIT` in [Section 5](#).

3.2 Submodules

A submodule is essentially an array of related operations that share a submodule ID. The purpose of submodules is for organization. That is, independent portions of a module do not need to coordinate operation IDs.

Enums are recommended for declaring submodule IDs, but not required. The only restriction on submodule IDs is that they are unique within a module and start from 0 with no gaps, as they are used for indexing arrays. The following is an example of declaring submodule IDs for a module that has GPIO and Timer operations:

```
001     /* Submodule IDs should be available to both secure and
002     * non-secure projects */
003     typedef enum
004     {
005         MY_SUBMODULE_GPIO,
006         MY_SUBMODULE_TIMER,
007         MY_SUBMODULE_END, /* One greater than max submodule */
008     } my_submodule_t;
```

Creating operations

The IDs should match the indexes used for submodule operation arrays. For example, setting up a module using these submodule IDs would look like so:

```
001      /* Arrays of operations */
002      mtb_srf_op_s_t* my_operations[MY_SUBMODULE_END] =
003      {
004          my_gpio_operations,
005          my_timer_operations,
006      };
007
008      /* Sizes of my_operations arrays */
009      size_t my_num_operations[] =
010      {
011          sizeof(my_gpio_operations) /
012          sizeof(my_gpio_operations[0]),
013          sizeof(my_timer_operations) /
014          sizeof(my_timer_operations[0]),
015      };
016
017      mtb_srf_module_s_t my_module =
018      {
019          .module_id = MTB_SRF_MODULE_USER,
020          .op_by_submod = my_operations,
021          .num_op_by_submod = my_num_operations,
022          .num_submod = MY_SUBMODULE_END
023      };
024
025      /* The secure project can then register this module so that
026      * the operations are available to the non-secure project */
027      mtb_srf_module_register(&cybsp_srf_context, &my_module);
```

The non-secure project will specify the module and submodule IDs when calling the SRF.

Note: *A common mistake is making operation structures or arrays static. Doing so may cause unpredictable behavior when they're accessed by pointer from another compilation unit in the SRF.*

Creating operations

3.3 Operations

Operations are defined by the `mtb_srf_op_s_t` type. This structure contains three groups of information: IDs used by the SRF to identify the operation, a function pointer to the implementation of the operation, and argument validation information.

```

001     typedef struct
002     {
003         // Module ID for this operation. Must be unique within
004         // this operation
005         uint8_t          module_id;
006         // Submodule ID for this operation
007         uint8_t          submodule_id;
008         // ID for this operation
009         uint8_t          op_id;
010         // If this operation requires modifying the hardware state
011         bool             write_required;
012         // Function pointer to the operation. May perform custom
013         // validation, if any, before executing request
014         mtb_srf_op_fn_t   impl;
015         // Required size of input_values
016         size_t            input_values_len;
017         // Required size of output_values
018         size_t            output_values_len;
019         // Minimum length of inVec pointer arguments needed (From
020         // First element)
021         size_t            input_len[MTB_SRF_MAX_IOVEC - 1];
022         // Whether an input should be copied into secure memory
023         // before processing. This should be set to true if
024         // validation decisions are made based on data in this
025         // argument. inVec[0] is always copied.
026         bool             needs_copy[MTB_SRF_MAX_IOVEC - 1];
027         // Minimum length of outVec pointer arguments needed (From
028         // First element)
029         size_t            output_len[MTB_SRF_MAX_IOVEC - 1];
030         // List of hardware resources on which non-secure is
031         // permitted to perform this operation. Permission
032         // validation will be bypassed if this is null.
033         mtb_srf_permission_s_t* allowed_rsc;
034         // Length of the `allowed_rsc` array
035         size_t            num_allowed;
036     } mtb_srf_op_s_t;

```

The first two elements should be familiar: module and submodule IDs. Operation IDs can be declared like submodule IDs. Similarly, their only restriction is that they're an index counting from zero and that they're unique relative to their submodule.

Creating operations

3.3.1 Operation implementation

The operation implementation is the functional part of the operation, stored in the `impl` member of the operation structure. Once all validation is complete, this is the function that the SRF will call.

All operation implementations must match the generic function prototype:

```

001     typedef cy_rslt_t (* mtb_srf_op_fn_t)
002                           (mtb_srf_input_ns_t* inputs_ns,
003                           mtb_srf_output_ns_t* outputs_ns,
004                           mtb_srf_invec_ns_t* inputs_ptr_ns,
005                           uint8_t inputs_ptr_cnt_ns,
006                           mtb_srf_outvec_ns_t* outputs_ptr_ns,
007                           uint8_t outputs_ptr_cnt_ns);

```

The `input_ns` and `output_ns` are documented in [Section 2.4](#). These are passed directly to various operation implementations duplicating the same cast from `ioVec` to `input/output` structure.

As the input/output structures are always `inVec[0]/outVec[0]`, the `ioVecs` arrays passed in `inputs_ptr_ns` and `outputs_ptr_ns` start at `inVec[1]/outVec[1]`. The `inputs_ptr_cnt_ns` and `outputs_ptr_cnt_ns` reflect this too. `ioVecs` are documented in [Section 2.3](#).

Operation implementations may also add custom validation logic on top of the default SRF validation (see [Section 3.3.3](#)). The SRF checks addresses and argument sizes, but not argument values. Custom validation logic is useful for restricting values, such as adding a minimum and maximum value on a clock divider update operation.

For a simple example, an operation to perform GPIO toggling:

```

001     cy_rslt_t my_gpio_toggle_operation_impl_s(
002                           mtb_srf_input_ns_t* inputs_ns,
003                           mtb_srf_output_ns_t* outputs_ns,
004                           mtb_srf_invec_ns_t* inputs_ptr_ns,
005                           uint8_t inputs_ptr_cnt_ns,
006                           mtb_srf_outvec_ns_t* outputs_ptr_ns,
007                           uint8_t outputs_ptr_cnt_ns)
008     {
009         /* Suppress unused argument warnings - this operation is
010          * simple and doesn't need them */
011         CY_UNUSED_PARAMETER(outputs_ns);
012         CY_UNUSED_PARAMETER(inputs_ptr_ns);
013         CY_UNUSED_PARAMETER(inputs_ptr_cnt_ns);
014         CY_UNUSED_PARAMETER(outputs_ptr_ns);
015         CY_UNUSED_PARAMETER(outputs_ptr_cnt_ns);
016
017         /* Pull arguments out of the input structure */
018         GPIO_PRT_Type* base = (inputs_ns->request).base;
019         uint32_t pin_num = (inputs_ns->request).sub_block;
020
021         /* Our operation can be as simple as a PDL call */
022         Cy_GPIO_Inv(base, pin_num);
023         return (cy_rslt_t)CY_GPIO_SUCCESS;
024     }

```

Creating operations

This operation benefits from the request structure including a base and sub-block already. A more complex operation may require more arguments passed in. All non-pointer input can be packed into the input structure's flexible array.

For example, a snippet of a SMIF operation implementation which requires three arguments passed in via the input structure's flexible array, and has a status to return in the output structure:

```

001      /* Declare local variables and memcpy into them. input_values
002      * is a uint8_t array - passing it directly will cut off
003      * values over 8-bits. */
004      cy_rslt_t status;
005      cy_pdl_smif_srf_mem_num_erase_sector_in_t input;
006      cy_pdl_smif_srf_status_out_t output;
007      cy_stc_smif_mem_context_t context =
008      {
009          .base = (SMIF_Type*)((inputs_ns->request).base),
010          .requires_secure_call = true,
011      };
012      status = mtb_srf_copy_input_value(&input, sizeof(input),
013          inputs_ns);
014      if (status != CY_RSLT_SUCCESS)
015          return status;
016      /* Similarly, store the result of the operation as a local
017      * variable that we memcpy out of */
018      output.op_res = Cy_SMIF_MemNumEraseSector(&context,
019          input.memNum, input.address, input.length);
020      status = mtb_srf_copy_output_value(outputs_ns, &output,
021          sizeof(output));

```

We recommend structures for packing arguments into flexible arrays. While it will work to memcpy all arguments into the `input_ns.input_values` and using `sizeof` for indexing, it is harder to maintain and introduces risk for the caller that packages these arguments and the operation that unpacks them to get out of sync.

3.3.2 Permissions and resources

Permissions and resources are part of non-secure argument validation, but are noteworthy enough to warrant a separate subsection. These are lists of resources which operations may be invoked on, defined by the `mtb_srf_permission_s_t` type:

```

001      typedef struct
002      {
003          /* Register base address for the hardware resource. If not
004          * applicable, set to NULL */
005          void*          base;
006          /* Sub-block index, if applicable. E.g. TCPWM counter
007          * index. If not applicable, set to 0 */
008          uint16_t       sub_block;
009          /* If true, operations that modify the hardware state are
010          * allowed */
011          bool           write_allowed;
012      } mtb_srf_permission_s_t;

```

Creating operations

Base and sub-block were already discussed in [Section 2.4](#), but in summary, the base is the base address of the peripheral in memory, if applicable. That is, TCPWM0 or SCB5 or similar. Sub-block is an index to further distinguish resources. A TCPWM counter would use a base and counter to identify the target hardware, while SCB5 would just use a base address, and CLK_HF3 would use only a sub-block since there is no base associated with this resource.

The `write_allowed` argument defines whether the hardware resource can be written to. This corresponds to the `write_required` in the operation structure. If the non-secure project submits a request for an operation with `write_required=true`, but the resource's permission struct has `write_allowed=false`, the SRF will reject the request with a security policy violation error. If a resource cannot be read or written, it should not be included in the permissions array. The resources listed do not have to be continuous or in any order.

Operations may have unique permissions arrays, share them with other operations, or skip them entirely by setting the `allowed_rsc` argument to NULL. The only requirement is that the `num_allowed` argument contains the count, or 0 if NULL.

Here is an example permissions array for GPIO pins, where the secure project exposes P0_0, P0_3, and P0_4, but only allows the first two to be updated by the non-secure project:

```

001      mtb_srf_permission_s_t my_gpio_permissions[] =
002      {
003          {
004              .base = GPIO_PRT0,
005              .sub_block = 0,
006              .write_allowed = true,
007          },
008          {
009              .base = GPIO_PRT0,
010              .sub_block = 3,
011              .write_allowed = true,
012          },
013          {
014              .base = GPIO_PRT0,
015              .sub_block = 4,
016              .write_allowed = false,
017          }
018      };

```

3.3.3 Non-secure argument validation

The remaining arguments of the operation structure are used by the SRF to perform argument validation.

`input_values_len` and `output_values_len` correspond to the input structure's and output structure's flexible arrays. These are exact values, not minimums. The maximum length of the flexible arrays is defined by `MTB_SRF_MAX_ARG_IN_SIZE` and `MTB_SRF_MAX_ARG_OUT_SIZE` macros declared in the BSP's `mtb_srf_config.h`. All configuration options in this file are user customizable directly. If larger flexible arrays are needed, the macro values should be increased. The SRF will also reject requests if their flexible arrays are bigger than these macros allow.

`input_len` and `output_len` are the minimum `ioVec` lengths required by the operation. There are no size restrictions on the `ioVec` sizes. These arrays have lengths one less than the max `ioVec` count. This is because

Creating operations

the first inVec and first outVec lengths are already covered by the size of the input/output structure plus `input_values_len/output_values_len`.

Lastly is `needs_copy`. This enables the SRF's caching feature for inVecs. This creates a local copy of the inVec in secure memory for the operation to use. InVec[0], or the input structure, is always cached to protect against run-time modification by the non-secure project. It is highly recommended to enable the caching feature when the data in an inVec affects validation decisions.

3.3.4 Operation arrays

All of the arguments described in these subsections fit into an operation struct. You can take all the snippets listed above and combine them into one entry of a GPIO submodule operation array:

```

001     mtb_srf_op_s_t my_gpio_operations[] =
002     {
003         {
004             .module_id = MTB_SRF_MODULE_USER,
005             .submodule_id = MY_SUBMODULE_GPIO,
006             .op_id = MY_SRF_OP_TOGGLE;,
007             .write_required = true,
008             .impl = my_gpio_toggle_operation_impl_s,
009             .input_values_len = 0U,
010             .output_values_len = 0U,
011             .input_len = {0, 0, 0},
012             .needs_copy = {false, false, false},
013             .output_len = {0, 0, 0},
014             .allowed_rsc = my_gpio_permissions,
015             .num_allowed = 3UL
016         },
017         /* ... etc ... */
018     };

```

Note: *GPIO is used as an example for its simplicity throughout this user guide. The native security awareness of the GPIO hardware should be used instead in a real-world application.*

Creating operations

Here is a more complex operation structure, which includes a variable-length buffer input argument:

```
001      {
002          .module_id = MTB_SRF_MODULE_PDL,
003          .submodule_id = CY_PDL_SECURE_SUBMODULE_SMIF,
004          .op_id = CY_PDL_SMIF_OP_MEM_WRITE,
005          .write_required = true,
006          .impl = cy_pdl_smif_srf_mem_num_write_impl_s,
007          .input_values_len = (sizeof(uint8_t) + sizeof(uint32_t)),
008          .output_values_len = sizeof(cy_en_smif_status_t),
009          // The first param is the tx_buffer and its length is
010          // part of the request so we are forced to use a
011          // minimum length here corresponding to 1 word
012          .input_len = {0x1, 0, 0},
013          // Don't copy the input buffer, it's not used for
014          // validation
015          .needs_copy = {false, false, false},
016          .output_len = {0, 0, 0},
017          .allowed_rsc = mtb_pdl_smif_srf_permissions,
018          .num_allowed = (sizeof(mtb_pdl_smif_srf_permissions)
019                          /sizeof(mtb_pdl_smif_srf_permissions[0]))
020      },
```

Calling operations

4 Calling operations

Non-secure projects request for operations to be executed via calls to `mtb_srf_submit_request`. This API accepts two `ioVec` arrays and the corresponding array lengths. The primary responsibility of the non-secure project is to allocate `ioVec`s and pack them with arguments. These arguments should be declared with awareness of where they are located in memory.

4.1 Request pools and argument packing

The SRF uses pools of memory for requests – empty buffers of shared memory that can be allocated and freed through the SRF. The purpose of these pools is to declare one section of shared memory for SRF requests to use instead of individual requests declaring their own shared memory.

There are two types of memory pools allocated for making SRF requests. The request pool, used to allocate `ioVec`s, input, and output structures from, is initialized directly via `mtb_srf_pool_init`. The packet pool is used for calls from the non-TrustZone core to the TrustZone core. Packets contain pointers to the `ioVec`s plus additional information for IPC processing. The SRF manages allocation and freeing from the packet pool. It is initialized as part of `_cybsp_ipc_srf_init` in `cybsp_init()`.

The BSP initializes a default request pool for the PDL to use. The non-secure project may use this pool or declare additional pools. It is required to use the same `MTB_SRF_MAX_ARG_IN_SIZE` and `MTB_SRF_MAX_ARG_OUT_SIZE` variables as these are used by the SRF to validate arguments.

```

001     #if !defined(CYBSP_DISABLE_SRF_INIT)
002     result = mtb_srf_pool_init(&cy_pdl_srf_default_pool,
003                               &cy_pdl_srf_default_pool_memory[0],
004                               MTB_SRF_POOL_SIZE,
005                               MTB_SRF_MAX_ARG_IN_SIZE,
006                               MTB_SRF_MAX_ARG_OUT_SIZE);
007     if (result != CY_RSLT_SUCCESS)
008     {
009         return result;
010     }
011     #if defined(COMPONENT_MW_MTB_IPC)
012     /* IPC must be initialized after peripheral clocks */
013     result = _cybsp_ipc_srf_init();
014     if (result != CY_RSLT_SUCCESS)
015     {
016         return result;
017     }
018     #endif /* defined(COMPONENT_MW_MTB_IPC) */
019     #endif /* !defined(CYBSP_DISABLE_SRF_INIT) */

```

The purpose of these pools is to have dedicated shared memory locations for using with the SRF. The expected use is for the non-secure project to declare `ioVec` pointers, give them to the pool to allocate memory for, populate them, pass them to the SRF via `mtb_srf_request_submit`, then unpack them and free the pool memory.

The following is an example of doing this to invoke the GPIO toggle operation is shown in [Section 3.3.4](#). Note all the matching module, submodule, and operation IDs.

Calling operations

```

001     cy_rslt_t result = CY_RSLT_SUCCESS;
002     mtb_srf_invec_ns_t *inVec = NULL;
003     mtb_srf_outvec_ns_t *outVec = NULL;
004
005     result = mtb_srf_pool_allocate(&cy_pdl_srf_default_pool,
006                                   &inVec, &outVec, WAITTIME);
007     CY_ASSERT(result == CY_RSLT_SUCCESS);
008
009     /* Pack arguments for identifying the GPIO toggle operation */
010     mtb_srf_input_ns_t *input =
011         (mtb_srf_input_ns_t *) (inVec[0].base);
012     mtb_srf_output_ns_t *output =
013         (mtb_srf_output_ns_t *) (outVec[0].base);
014     input->request.module_id = MTB_SRF_MODULE_USER;
015     input->request.submodule_id = MY_SUBMODULE_GPIO;
016     input->request.op_id = MY_SRF_OP_TOGGLE;
017     input->request.base = GPIO_PRT0;
018     input->request.sub_block = 3UL;
019     /* This operation doesn't use the flexible arrays or other
020      * ioVecs */
021     input->len = 0UL;
022     output->len = 0UL;
023     inVec[0].len = sizeof(mtb_srf_input_ns_t) + input->len;
024     outVec[0].len = sizeof(mtb_srf_output_ns_t) + output->len;
025     result = mtb_srf_request_submit(inVec, 0x1, outVec, 0x1);
026     CY_ASSERT(result == CY_RSLT_SUCCESS);
027
028     /* Output values are passed in by value. Make a copy before
029      * freeing the ioVecs */
030     memcpy(&result, (&(output->output_values[0])), sizeof(result));
031     CY_ASSERT(result == CY_RSLT_SUCCESS);
032
033     result = mtb_srf_pool_free(&cy_pdl_srf_default_pool, inVec,
034                               outVec);
035     CY_ASSERT(result == CY_RSLT_SUCCESS);

```

The memory region allocated for the pool to use should be aligned to DCache lines and in shared memory, when applicable. The pool allocation API, `mtb_srf_pool_allocate`, will automatically pad to ensure DCache alignment. More on alignment and shared memory in the next section.

4.2 Shared memory and DCache alignment

When passing arguments between cores or security states, it's important to ensure that the data is accessible to both the sender and the receiver. For requests between partitions of a TrustZone-enabled core, it's straightforward – simply ensure data is in the TrustZone core's non-secure memory. For cross-core, it requires informing the linker to use a shared memory region.

The MTB project environment offers macros to place variables in specific memory regions. These are toolchain agnostic. To place a variable in the shared memory region:

```

001     CY_SECTION_SHAREDMEM uint32_t my_pool_memory[size];

```

Calling operations

Generally, all data passed cross-core should be in shared memory. However, there is an exception. On PSoC™ Edge devices, the CM55 core has dedicated memory regions Instruction Tightly Coupled Memory (ITCM) and Data Tightly Coupled Memory (DTCM). Variables declared on the CM55 will default to the DTCM region. This region is not covered by an MPC, and is accessible by both cores through aliases. The SRF will automatically convert DTCM addresses to their CM33-accessible address aliases, allowing the CM55 to pass in local memory addresses as ioVec bases. Note that while this is allowed, it's not recommended. DTCM access from other cores is slower than shared memory access.

The automatic DTCM memory aliasing does not apply to the pools – pool memory should still be declared in shared memory.

When DCache is present and enabled, the SRF will clean and invalidate the ioVecs, input and output structures, and data that the ioVecs point to. However, to achieve this, the SRF requires that all variables are aligned to the DCache cache line size. The request and packet pool allocate DCache-aligned ioVecs and packets by default. This means the user application's responsibility is only for data pointed to by ioVecs. The SRF offers a macro for DCache alignment, `_MTB_SRF_DATA_ALIGN`.

```

001      /* Example of a PSoC™ Edge SRF argument packing from the
002      * CM55. */
003      /* A generic array will default to DTCM memory */
004      _MTB_SRF_DATA_ALIGN uint8_t dtcm_array[100];
005      /* Using shared memory is also valid */
006      _MTB_SRF_DATA_ALIGN CY_SECTION_SHAREDMEM uint8_t
007                          shared_mem_array[100];
008
009      /* ... in some later function ... */
010
011      inVec[1].base = &shared_mem_array[0];
012      inVec[1].len = (sizeof(shared_mem_array)/
013                     sizeof(shared_mem_array[0]));
014      inVec[2].base = &dtcm_array[0];
015      inVec[2].len = (sizeof(dtcm_array)/sizeof(dtcm_array[0]));
016
017      inVec[0]->len = sizeof(mtb_srf_input_ns_t) + input->len;
018      outVec[0]->len = sizeof(mtb_srf_output_ns_t) + output->len;
019      result = mtb_srf_request_submit(inVec, 0x3, outVec, 0x1);
020      CY_ASSERT(result == CY_RSLT_SUCCESS);

```

Note: *The IAR toolchain requires that the `CY_SECTION_SHAREDMEM` precede the `_MTB_SRF_DATA_ALIGN`. If the data alignment comes first, this toolchain may not place the variable in the correct memory.*

Customization

5 Customization

The SRF allows a fair amount of customization. The makefile define `CYBSP_DISABLE_SRF_INIT` disables all the BSP SRF initialization and setup and default request submission APIs. This is intended for when the application wishes to customize the SRF initialization. It is recommended to review the existing SRF initialization performed in the device's BSP and build upon that for customization.

This snippet shows the default request submission APIs on a PSOC™ Edge device.

```

001     #if defined(COMPONENT_SECURE_DEVICE)
002     //-----
003     // mtb_srf_request_submit
004     //-----
005     cy_rslt_t __attribute__((cmse_nonsecure_entry))
006         mtb_srf_request_submit(
007             mtb_srf_invec_ns_t* inVec_ns, uint8_t inVec_cnt_ns,
008             mtb_srf_outvec_ns_t* outVec_ns, uint8_t outVec_cnt_ns)
009     {
010         return mtb_srf_request_execute(&cybsp_srf_context,
011                                         inVec_ns, inVec_cnt_ns,
012                                         outVec_ns, outVec_cnt_ns);
013     }
014     #endif /* defined(COMPONENT_SECURE_DEVICE) */
015     #if defined(COMPONENT_NON_SECURE_DEVICE) &&
016         (CY_SYSTEM_CPU_M55)
017     //-----
018     // mtb_srf_request_submit
019     //-----
020     cy_rslt_t mtb_srf_request_submit(
021         mtb_srf_invec_ns_t* inVec_ns, uint8_t inVec_cnt_ns,
022         mtb_srf_outvec_ns_t* outVec_ns, uint8_t outVec_cnt_ns)
023     {
024         #if defined(MTB_SRF_SUBMIT_USE_IPC) &&
025             defined(COMPONENT_MW_MTB_IPC)
026         return mtb_srf_ipc_request_submit(
027             &cybsp_mtb_srf_client_context,
028             inVec_ns, inVec_cnt_ns,
029             outVec_ns, outVec_cnt_ns);
030         #else /* defined(MTB_SRF_SUBMIT_USE_IPC) &&
031             defined(COMPONENT_MW_MTB_IPC) */
032         CY_UNUSED_PARAMETER(inVec_ns);
033         CY_UNUSED_PARAMETER(inVec_cnt_ns);
034         CY_UNUSED_PARAMETER(outVec_ns);
035         CY_UNUSED_PARAMETER(outVec_cnt_ns);
036         CY_ASSERT(false);
037         return MTB_SRF_ERR_UNKNOWN_OPERATION;
038         #endif /* defined(MTB_SRF_SUBMIT_USE_IPC) &&
039             defined(COMPONENT_MW_MTB_IPC) */
040     }
041
042     #endif /* defined(COMPONENT_NON_SECURE_DEVICE) &&
043         (CY_SYSTEM_CPU_M55) */

```

Customization

The default request submission function implementations may also be disabled alone through `MTB_SRF_CUSTOM_REQUEST_SUBMIT`. There are two versions of `mtb_srf_request_submit` available. The first is the NSC, used by the TrustZone core to call from the non-secure side to the secure side. The second is used by the non-TrustZone core, a wrapper for `mtb_srf_ipc_request_submit` which packages the request into a packet and puts it in an IPC mailbox for the non-secure side of the TrustZone core to process. The two APIs are shown below, they are defined in the BSP's `cybsp.c` file.

The requirements for custom implementations are straightforward:

- The prototypes must match
- The NSC must ultimately call `mtb_srf_request_execute`
- The pseudo-NSC must ultimately call `mtb_srf_ipc_request_submit`

The remaining customization comes from the `mtb_srf_config.h` file in the BSP:

- The input and output struct variable array maximum sizes are defined here via `MTB_SRF_MAX_ARG_IN_SIZE` and `MTB_SRF_MAX_ARG_OUT_SIZE`. These sizes affect both the default pool initialized in the BSP and the logic inside the SRF itself.
- The scratch buffer size. This is the buffer used to cache the input and output structures and any inVecs marked via the operation struct member `needs_copy`.
- The number of requests that the default pool can allocate at a time is controlled by `MTB_SRF_POOL_SIZE`. Too small of a pool may result in the error `MTB_SRF_ERR_ALLOCATE_FREE` when trying to allocate requests.
- The `MTB_SRF_IPC_SERVICE_TIMEOUT_US` is the timeout for SRF operations called from the non-TrustZone core. The default value is to wait forever for the TrustZone core to service the request.
- Lastly are the module count and module IDs. These are discussed in [Section 3.1](#).

These options allow configuring and disabling pieces of the SRF. There is a broader makefile define to disable all of the SRF: `CY_SRF_DISABLE`. This is intended for applications which do not use the SRF. The SRF asset is a default dependency of many BSPs and cannot easily be removed. This also automatically disables PDL-SRF integration, akin to `CY_PDL_ENABLE_SECURE_AWARE=0`. See [Section 2.6.1](#) for further details on PDL integration.

Revision history

Revision history

Document version	Date of release	Description of changes
**	2025-09-30	New document.

Trademarks

All referenced product or service names and trademarks are the property of their respective owners.

Edition 2025-09-30

Published by

Infineon Technologies AG
81726 Munich, Germany

© 2025 Infineon Technologies AG.
All Rights Reserved.

Do you have a question about this document?

Email:

erratum@infineon.com

002-42149 Rev. **

Important Notice

Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.

For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability.

Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.

The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.

Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury. However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer.

Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG). If the Product includes security features:

Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.

If this document includes or references software:

The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved. Therefore, you may use the software only as provided in the software license agreement accompanying the software.

If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited. For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit <https://www.infineon.com>